

KNOWLEDGE CORPUS • ARCHITECTURE SPECIFICATION

# Docker & Container Runtime Architecture

A Structured Technical Reference on Linux Kernel Primitives, OCI Standards, Layered Storage, Container Networking, and Runtime Mechanics

Doc ID: GL-CORPUS-01

Domain: Cloud Infrastructure

Topic: Container Runtimes

Target: Graph RAG System

## 1. Prerequisites & Fundamental Knowledge

To fully understand container runtime architecture, an engineer must be familiar with fundamental operating system and networking concepts. Containers do not run on specialized physical hardware; they run on standard Linux operating systems hosted on physical bare-metal servers or cloud virtual machines such as Amazon Elastic Compute Cloud (Amazon EC2) instances.

The reader should possess a foundational understanding of the following basic concepts before reading this technical document:

- Linux Operating System & Kernel:** The Linux kernel is the core program that manages the hardware resources of a computer, including the Central Processing Unit (CPU), Random Access Memory (RAM), storage disks, and network interface cards. Applications interact with the Linux kernel through system calls (syscalls).
- Processes & Threads:** In Linux, a process is an executing instance of a computer program with its own address space, file descriptors, and execution state. Every process is assigned a unique Process Identifier (PID).
- Virtual Machines (VMs) vs. Containers:** A Virtual Machine emulates an entire physical computer, requiring a Hypervisor (such as KVM, VMware ESXi, or AWS Nitro) and a complete guest operating system. A Container is a regular Linux process that runs directly on the host Linux kernel, sharing the kernel with other containers while remaining logically isolated.
- Basic Computer Networking:** Familiarity with IP addresses, Subnets, Classless Inter-Domain Routing (CIDR), Network Address Translation (NAT), Ethernet frames, and OSI Model layers (Layer 2 Data Link, Layer 3 Network, Layer 4 Transport, and Layer 7 Application).
- Cloud Infrastructure:** Basic familiarity with public cloud platforms such as Amazon Web Services (AWS), managed virtual machine instances like Amazon EC2, and managed container orchestrators such as Amazon Elastic Kubernetes Service (Amazon EKS).

## 2. Fundamental Containerization Concepts & Terminology

Before exploring low-level runtime interactions, we must establish clear, unambiguous definitions for the core building blocks of container technology. Each concept listed below forms a critical entity in the container runtime ecosystem.

### Concept: Docker & Docker Engine

- Beginner-Friendly Definition:** Docker is a commercial and open-source containerization platform and developer tooling suite that packages software into standardized, portable container units. Docker Engine is the core client-server application consisting of the Docker CLI, a REST API daemon (dockerd), and backend integration with containerd.
- Why It Exists:** Prior to Docker, creating containers required manual, complex Linux kernel syscalls and scripts. Docker popularized containerization by providing an easy-to-use CLI, standardized Dockerfile build automation, and image distribution tooling.
- How It Works:** Docker accepts user commands from the Docker CLI, manages container builds via BuildKit, coordinates storage volumes and network bridges, and delegates low-level runtime tasks to containerd and runc.
- Real-World Cloud Example:** Developers use Docker CLI on local machines to build container images, run local microservice tests, and push images to AWS Elastic Container Registry (ECR).
- Related Concepts:** Docker CLI, dockerd, containerd, runc, Dockerfile, Container Image.

### Concept: Container

**Beginner-Friendly Definition:** A container is an isolated Linux process running on a shared host operating system, packaged with all the code, runtime libraries, and dependencies it needs to execute reliably across different environments.

**Why It Exists:** To eliminate the "works on my machine" problem by ensuring applications execute in identical environments across development laptops, testing pipelines, and production cloud clusters.

**How It Works:** The host Linux kernel uses Namespaces to hide system resources from the container process and Control Groups (cgroups) to limit the amount of CPU, memory, and disk I/O the process can consume.

**Real-World Cloud Example:** A Python microservice deployed inside an Amazon ECS task or an Amazon EKS Pod running on an Amazon EC2 instance.

**Related Concepts:** Linux Namespaces, Control Groups, Container Images, Virtual Machines, Docker Engine.

### Concept: Container Image

**Beginner-Friendly Definition:** A container image is a read-only, unchangeable template containing the application code, system libraries, configuration files, and default execution instructions needed to create a running container.

**Why It Exists:** To provide a reproducible, portable, and version-controlled software distribution artifact that can be stored in container registries and downloaded on demand.

**How It Works:** An image consists of multiple read-only filesystem layers stacked on top of each other. Each layer represents a modification made during the build process, identified by a cryptographic SHA-256 hash.

**Real-World Cloud Example:** An application image pushed to Amazon Elastic Container Registry (Amazon ECR) and pulled by worker nodes in an Amazon EKS cluster.

**Related Concepts:** OCI Image Specification, OverlayFS, Layered Filesystem, Dockerfile, Container Registry.

### Concept: Open Container Initiative (OCI)

**Beginner-Friendly Definition:** The Open Container Initiative (OCI) is an open industry governance structure and standards body that defines vendor-neutral specifications for container formats and container runtimes.

**Why It Exists:** To prevent vendor lock-in and ensure that container images created by one tool (like Docker or Buildah) can be executed by any compliant runtime (like containerd, runc, or CRI-O).

**How It Works:** OCI maintains two foundational standards: the OCI Image Specification (defining image format, layer archives, and manifests) and the OCI Runtime Specification (defining how a runtime must unpack, configure, and execute a container process).

**Real-World Cloud Example:** AWS Fargate runs OCI-compliant container images without requiring the traditional Docker desktop client or daemon.

**Related Concepts:** OCI Image Spec, OCI Runtime Spec, runc, containerd, Docker Engine.

### Concept: Container Runtime Interface (CRI)

**Beginner-Friendly Definition:** The Container Runtime Interface (CRI) is a standardized plugin interface and gRPC protocol that allows Kubernetes to communicate with different container runtimes without recompiling the Kubernetes source code.

**Why It Exists:** Originally, Kubernetes was hard-coded to work only with the Docker daemon. CRI was introduced to decouple Kubernetes from specific runtime implementations, allowing Kubernetes to use lightweight runtimes like containerd or CRI-O directly.

**How It Works:** The Kubernetes node agent (Kubelet) sends gRPC API requests over a local Unix domain socket to a CRI-compliant runtime daemon (such as containerd) to pull images, run containers, and monitor container status.

**Real-World Cloud Example:** Modern Amazon EKS worker nodes use containerd via CRI, completely bypassing the legacy Docker daemon.

**Related Concepts:** Kubernetes Kubelet, containerd, CRI-O, gRPC, Unix Domain Socket.

### 3. Linux Kernel Isolation Primitives: Namespaces and cgroups

Containers are not magic abstractions; they are standard Linux processes whose boundaries are enforced entirely by the Linux kernel. The kernel provides two essential mechanisms: **Linux Namespaces** for isolation and **Control Groups (cgroups)** for resource management.

#### 3.1 Linux Namespaces: Isolating System Visibility

A Linux Namespace wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of that resource. When a process runs inside a namespace, it cannot see or interact with resources belonging to other namespaces or the host root namespace.

The Linux kernel provides seven primary namespaces used by container runtimes:

- **PID Namespace (Process ID Isolation):** Virtualizes process IDs. Inside a container's PID namespace, the primary application process is assigned PID 1. PID 1 is responsible for initializing the environment, handling operating system signals (such as SIGTERM and SIGKILL), and reaping terminated child processes (preventing zombie processes). On the host operating system, the exact same process is assigned a standard high PID (such as PID 48291).
- **Mount Namespace (Filesystem Isolation):** Isolates the filesystem mount points visible to a process. When combined with the Linux `pivot_root` system call, the Mount Namespace ensures that the container process sees its own isolated directory structure as the root directory (`/`), preventing the container from accessing host files.
- **Network Namespace (Network Stack Isolation):** Isolates physical and virtual network devices, IP routing tables, firewall filtering rules (iptables and nftables), port numbers, and socket lists. Each network namespace receives its own independent loopback interface (`127.0.0.1`) and virtual Ethernet interfaces.
- **IPC Namespace (Inter-Process Communication Isolation):** Isolates System V IPC mechanisms and POSIX message queues. Processes in different IPC namespaces cannot share shared memory segments or send inter-process messages to each other.
- **UTS Namespace (UNIX Timesharing System Isolation):** Isolates the system hostname and Network Information Service (NIS) domain name. This allows each container to have its own unique hostname (e.g., `web-server-pod-1`) without modifying the host machine's hostname.
- **User Namespace (User and Group ID Mapping):** Maps user IDs (UIDs) and group IDs (GIDs) inside the container to different UIDs and GIDs on the host system. This allows a process to run with root privileges (UID 0) inside the container while being mapped to an unprivileged user (e.g., UID 10001) on the host system.
- **Cgroup Namespace (Control Group View Isolation):** Isolates the process view of its own control group hierarchy in `/proc/self/cgroup`, preventing the container from observing the host's overall cgroup layout.

#### 3.2 Control Groups (cgroups): Limiting Resource Consumption

While Linux Namespaces determine what a process can *see*, Control Groups (cgroups) determine what a process can *use*. A control group is a Linux kernel feature that allows system administrators and container runtimes to allocate, meter, prioritize, and limit hardware resource consumption (CPU, RAM, disk I/O, network bandwidth, and process counts) for a collection of processes.

Modern cloud distributions use **cgroups v2**, which replaces the legacy multi-hierarchy model of cgroups v1 with a unified single-hierarchy tree. The major resource controllers include:

- **Memory Controller (`memory.max`):** Sets a hard limit on the total RAM a container can allocate. If a container exceeds its configured `memory.max` boundary, the Linux kernel Out-Of-Memory (OOM) Killer terminates the process with **Exit Code 137**.
- **CPU Controller (`cpu.max`):** Enforces CPU bandwidth limits using the Linux Completely Fair Scheduler (CFS). The runtime specifies a quota and a period (e.g., 50,000 microseconds per 100,000 microseconds period), throttling CPU cycles when the quota is reached to prevent CPU starvation on shared multi-tenant hosts.

- **PID Controller (`pids.max`):** Restricts the maximum number of processes and threads that can be spawned inside the control group. This controller protects the host against "fork bombs" and malicious denial-of-service attempts.
- **I/O Controller (`io.max`):** Limits read and write operations per second (IOPS) and throughput (bytes per second) on block storage devices, preventing disk saturation by rogue containers.

| Kernel Primitive         | Primary Function        | Kernel Mechanism                                  | Operational Failure If Misconfigured            |
|--------------------------|-------------------------|---------------------------------------------------|-------------------------------------------------|
| <b>PID Namespace</b>     | Process isolation       | Virtual PID mapping via <code>CLONE_NEWPID</code> | Zombie process leaks; host PID conflicts        |
| <b>Mount Namespace</b>   | Filesystem isolation    | Mount point isolation via <code>pivot_root</code> | Unauthorized access to host root filesystem     |
| <b>Network Namespace</b> | Network stack isolation | Virtual interfaces & routing tables               | Port conflicts; cross-tenant packet sniffing    |
| <b>User Namespace</b>    | Privilege separation    | UID/GID translation tables                        | Container breakout yields full host root access |
| <b>Memory Cgroup</b>     | RAM usage limitation    | Kernel accounting via <code>memory.max</code>     | OOM panic on host; node starvation              |
| <b>CPU Cgroup</b>        | CPU cycle allocation    | CFS bandwidth quota via <code>cpu.max</code>      | Noisy-neighbor CPU exhaustion                   |

## 4. Layered Storage, OverlayFS, and Copy-on-Write Mechanics

Container storage architectures must balance two competing goals: instant container startup times and minimal disk space consumption. If every container required a complete copy of a 1 GB operating system root filesystem, launching 100 containers would consume 100 GB of storage and take minutes to copy. Container runtimes solve this challenge through **Union Filesystems** and **Copy-on-Write (CoW)** technology.

### 4.1 Understanding OverlayFS (Overlay2)

The standard storage driver used by modern Linux container engines is **Overlay2**, which is built on the Linux kernel **OverlayFS** filesystem. OverlayFS takes multiple distinct directories on the host disk and presents them to the user as a single unified, combined directory tree.

OverlayFS organizes filesystems into four distinct functional directories:

- **LowerDir (Read-Only Image Layers):** A collection of stacked, read-only directories containing the files from the container image. LowerDir directories are completely immutable. When 50 containers are launched from the same Ubuntu image, all 50 containers share the exact same LowerDir files on disk without duplicating a single byte.
- **UpperDir (Read-Write Container Layer):** A single read-write directory created specifically for a running container instance. When the container creates a new file, modifies an existing file, or deletes a file, all changes are written exclusively to UpperDir. When the container is deleted, UpperDir is destroyed.
- **WorkDir (Kernel Staging Directory):** An internal temporary workspace used by the Linux kernel to prepare and validate file updates atomically before moving them into UpperDir.
- **Merged Directory (Container Root Filesystem):** The combined virtual mount point generated by OverlayFS. The container runtime sets this Merged directory as the root filesystem (`/`) of the container's Mount Namespace. The running container only sees this unified Merged view.

```

+-----+
|           MERGED VIEW (Container sees this as / )           |
+-----+
|                                     ^                         |
|                                     | (Union Mount)            |
+-----+
| UPPERDIR: Read-Write Ephemeral Layer (Modifications & New Files) |
+-----+
| LOWERDIR 3: Read-Only Application Code (SHA: a4f91b...)      |
+-----+
| LOWERDIR 2: Read-Only Runtime / Python (SHA: 8c3e01...)      |
+-----+
| LOWERDIR 1: Read-Only Base OS / Debian (SHA: d71a44...)      |
+-----+

```

## 4.2 Copy-on-Write (CoW) in Action

The Copy-on-Write (CoW) optimization strategy ensures that files are only copied to the read-write layer when they are actually modified:

1. **Reading a File:** When a container process reads a file (e.g., `/etc/hosts`), OverlayFS checks UpperDir first. If the file is not in UpperDir, it searches downward through the LowerDir layers and reads the file directly from the immutable base image.
2. **Modifying an Existing File:** When a container process attempts to modify a file that exists only in LowerDir, OverlayFS intercepts the write operation. It copies the entire original file from the read-only LowerDir up into the read-write UpperDir. The process then applies the changes to the copy in UpperDir. The original file in LowerDir remains untouched.
3. **Creating a New File:** When a container creates a brand-new file, the file is written directly into UpperDir.
4. **Deleting a File:** When a container deletes a file that exists in LowerDir, OverlayFS creates a special placeholder file called a *whiteout file* in UpperDir. The whiteout file masks the lower file, making it invisible in the Merged view without deleting the underlying read-only image layer.

**Cloud Performance Consideration:** Because Copy-on-Write copies the entire file into UpperDir upon modification, applications that perform high-frequency random writes to large files (such as MySQL, PostgreSQL, or MongoDB) suffer significant performance degradation on Overlay2. In production cloud deployments on AWS EC2 or EKS, databases must use external Amazon Elastic Block Store (Amazon EBS) persistent volume mounts, which bypass Overlay2 entirely.

## 5. Container Runtime Architecture: High-Level and Low-Level Engines

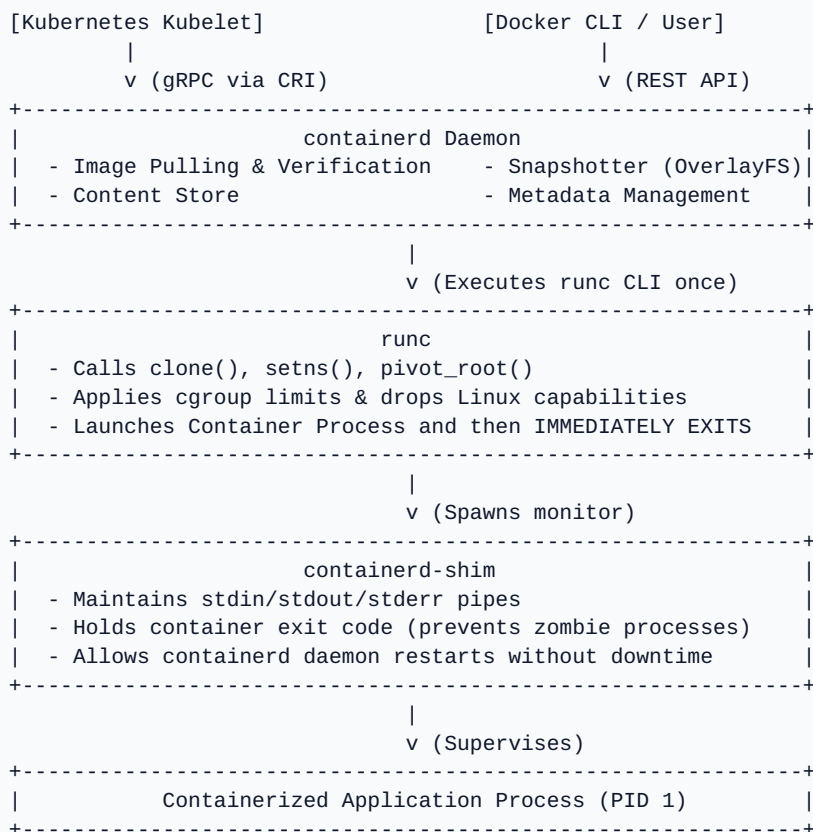
Modern container architecture is strictly modular and decomposed into distinct layers. The monolithic architecture of early Docker engines has been separated into specialized high-level and low-level runtime components governed by OCI specifications.

### 5.1 The Architecture Stack: Docker, containerd, runc, and containerd-shim

When an engineer runs a command like `docker run -d -p 80:80 nginx`, or when a Kubernetes Kubelet starts a Pod, multiple software components work together in a coordinated pipeline:

1. **Docker Engine (CLI & Daemon):** Docker provides user-friendly developer tooling, the Docker CLI, volume management, build systems (BuildKit), and local image management. Docker Engine acts as an orchestration client that delegates runtime operations to **containerd**.
2. **containerd (High-Level Runtime Daemon):** containerd is an industry-standard container runtime daemon managed by the Cloud Native Computing Foundation (CNCF). containerd is responsible for pulling container images from remote registries, verifying cryptographic digests, unpacking image layers onto disk snapshotters, managing network namespaces, and supervising container lifecycles. containerd implements the Kubernetes Container Runtime Interface (CRI).

3. **runc (Low-Level OCI Runtime):** runc is a lightweight command-line tool written in Go that directly interacts with the Linux kernel. containerd invokes runc with an OCI `config.json` file and an unpacked root filesystem directory. runc executes low-level system calls (`clone()`, `unshare()`, `setns()`, and `pivot_root`) to create the namespaces, configure cgroup limits, set up seccomp filters, and start the application process.
4. **containerd-shim (Process Monitor):** Once runc launches the container process, runc immediately terminates and exits. containerd spawns a tiny, lightweight background process called **containerd-shim** to sit directly above the container process. The containerd-shim stays alive for the entire lifespan of the container.



## 5.2 Why containerd-shim is Critical for Production Stability

The containerd-shim serves three vital operational purposes in cloud infrastructure:

- **Daemonless Containers (Zero-Downtime Maintenance):** If the containerd daemon or Docker daemon crashes or requires a software upgrade, running containers do not stop. Because containerd-shim acts as the parent process of the container, the container continues running completely unaffected during daemon restarts.
- **I/O Stream Handling:** containerd-shim keeps the standard input (stdin), standard output (stdout), and standard error (stderr) file descriptors open. If containerd restarts, it reconnects to the shim's I/O pipes without losing application log streams.
- **Exit Status Reporting & Zombie Reaping:** When a container process terminates, containerd-shim captures its exit code and holds it until containerd reads it. This prevents the terminated process from becoming an un-reaped zombie process on the host operating system.

## 6. Container Networking Architecture & Traffic Flows

Container networking enables isolated container processes to communicate with other containers on the same host, with external cloud services, and with clients on the public internet.

## 6.1 Virtual Ethernet (veth) Pairs and Bridge Networking

In standard standalone container environments, networking relies on **Linux Bridge Networking** and **Virtual Ethernet (veth) pairs**:

- **The veth Pair (Virtual Wire):** A veth pair is a pair of interconnected virtual network interfaces created by the Linux kernel. Whatever packet enters one end of the pair immediately emerges from the other end. When a container is created, the runtime creates a veth pair (e.g., veth1234 and eth0). It places eth0 inside the container's private Network Namespace and keeps veth1234 in the host's default root Network Namespace.
- **The Linux Bridge (docker0 / cbr0):** A software bridge is an OSI Layer 2 virtual switch operating inside the host kernel. The host end of the veth pair (veth1234) is attached to the bridge (such as docker0). The bridge connects all local container veth interfaces together, assigning each container an IP address from a private subnet (e.g., 172.17.0.0/16).

## 6.2 Network Address Translation (NAT) and Packet Routing

Because container IP addresses belong to private, non-routable internal subnets, communication outside the host requires **Network Address Translation (NAT)** managed by Linux iptables or nftables:

- **Outbound Traffic (Egress / SNAT):** When a container initiates a connection to an external endpoint (such as an AWS RDS database or an external API), the Linux kernel applies a **Source NAT (SNAT)** / IP MASQUERADE rule. The kernel replaces the container's private source IP (172.17.0.2) with the host machine's public or VPC network IP address (e.g., 10.0.1.50) before sending the packet onto the physical network.
- **Inbound Traffic (Ingress / DNAT / Port Forwarding):** When a container port is published to the outside world (such as `docker run -p 8080:80 nginx`), the runtime inserts a **Destination NAT (DNAT)** rule into the host kernel's PREROUTING iptables chain. When an external client sends a request to host port 8080, the host kernel intercepts the packet, rewrites the destination IP and port to the container's internal IP (172.17.0.2:80), and forwards the packet across the bridge.

```
[External Client] ---> (Port 8080) ---> [Host Physical Interface eth0 (10.0.1.50)]
                                     |
                                     v (iptables PREROUTING DNAT)
                                [Linux Bridge docker0 (172.17.0.1)]
                                     |
                                     v (veth pair link)
                                [Container eth0 Interface (172.17.0.2:80)]
```

## 7. Container Security, Hardening & Isolation Boundaries

Because containers share the host Linux kernel, securing a containerized environment requires defense-in-depth across multiple kernel-level access control frameworks.

### 7.1 Linux Capabilities

Traditionally, Linux processes are either unprivileged (regular user) or fully privileged (root superuser UID 0). The Linux kernel divides traditional root privileges into distinct, granular units called **Capabilities**. For example, CAP\_NET\_BIND\_SERVICE allows a process to bind to privileged ports below 1024, while CAP\_SYS\_ADMIN grants broad administrative powers.

By default, container runtimes drop dangerous capabilities from root container processes (stripping CAP\_SYS\_ADMIN, CAP\_SYS\_RAWIO, CAP\_SYS\_MODULE, and CAP\_SYS\_BOOT). In hardened production environments, teams should apply the principle of least privilege: drop all capabilities by default (`--cap-drop=ALL`) and explicitly add back only the specific capabilities required (such as `--cap-add=NET_BIND_SERVICE`).

## 7.2 Secure Computing Mode (Seccomp)

The Linux kernel provides over 300 distinct system calls (syscalls). Many system calls (such as `reboot`, `kexec_load`, `mount`, and `ptrace`) are never needed by standard web applications and could be exploited to compromise the host kernel.

**Seccomp (Secure Computing Mode)** is a Linux kernel security facility that filters system calls executed by a process. An OCI runtime applies a JSON-defined seccomp filter when launching a container. By default, containerd and Docker apply a seccomp filter that blocks more than 40 dangerous system calls while allowing standard application operations, mitigating kernel-level zero-day vulnerabilities.

## 7.3 Mandatory Access Control (AppArmor and SELinux)

Mandatory Access Control (MAC) frameworks enforce fine-grained security policies on system resources regardless of file ownership permissions:

- **AppArmor (Ubuntu/Debian):** Path-based access control. AppArmor profiles define which specific files, directories, and network sockets a container process is permitted to read, write, or execute.
- **SELinux (RHEL/CentOS/Amazon Linux 2023):** Label-based access control. SELinux assigns security contexts and labels (e.g., `container_t`) to processes and files, ensuring that even if a container process runs as root, it cannot access files labeled for the host operating system (`system_u:object_r:etc_t`).

## 7.4 Rootless Containers

In standard configurations, the container runtime daemon runs as the host root user. In a **Rootless Container** configuration, the entire container management engine (including containerd and runc) executes entirely within an unprivileged user namespace without host root permissions. If an attacker manages to break out of a rootless container, they obtain only unprivileged user access on the host, preventing host takeover.

# 8. Practical Operational Troubleshooting

Diagnosing container failures in cloud environments requires understanding the exact exit codes and kernel-level signals emitted during container termination.

| Exit Code / Error                           | Root Cause                                                                                                             | Diagnostic Command                                                                                        | Remediation Strategy                                                                          |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Exit Code 137 (OOMKilled)</b>            | Process memory exceeded cgroup <code>memory.max</code> ceiling; kernel OOM killer sent <code>SIGKILL</code> (128 + 9). | <code>dmesg -T   grep -i oom</code> or inspect cgroup events at <code>/sys/fs/cgroup/memory.events</code> | Increase container memory limits or optimize application heap usage.                          |
| <b>Exit Code 139 (Segmentation Fault)</b>   | Process attempted illegal memory access; binary sent <code>SIGSEGV</code> (128 + 11).                                  | <code>journalctl -u containerd</code> or inspect core dumps                                               | Verify CPU architecture match (e.g., AMD64 vs. ARM64/Graviton) and check dynamic libraries.   |
| <b>Exit Code 143 (Graceful Termination)</b> | Process received <code>SIGTERM</code> signal (128 + 15) from orchestrator during scaling or deployment.                | Check orchestrator deployment events (e.g., <code>kubectl describe pod</code> )                           | Normal operational state; ensure application handles graceful shutdown within timeout period. |
| <b>Zombie Process Leak</b>                  | Container PID 1 did not reap terminated child processes via <code>SIGCHLD</code> handler.                              | <code>ps aux   grep 'Z'</code> on host or inside container                                                | Use a lightweight init process like <code>tini</code> as container entrypoint.                |
| <b>OverlayFS Disk Exhaustion</b>            | Application wrote massive ephemeral logs or temp files directly into <code>UpperDir</code> .                           | <code>df -h</code> and <code>du -sh /var/lib/docker/overlay2/*</code>                                     | Mount dedicated volume storage (EBS / tmpfs) for high-write paths.                            |

## 9. Explicit Architectural Relationships & Dependencies

---

This section explicitly documents the structural dependencies and multi-hop interactions between technologies in the container ecosystem. These relationships define how components communicate in production cloud platforms.

- **Docker Engine uses containerd:** Docker delegates core container lifecycle, image distribution, and snapshot management to containerd.
- **containerd uses runc:** containerd invokes the runc CLI to configure kernel namespaces and launch container processes according to the OCI Runtime Specification.
- **containerd spawns containerd-shim:** containerd creates a containerd-shim process for every container to monitor the process and maintain open I/O pipes.
- **Kubernetes Kubelet communicates with containerd through CRI:** The Kubelet sends gRPC requests over a local Unix domain socket to containerd via the Container Runtime Interface.
- **Containers use Linux Namespaces for isolation:** The container runtime configures PID, Mount, Network, IPC, UTS, and User namespaces to isolate process visibility.
- **Containers use Control Groups (cgroups) for resource limits:** The Linux kernel CFS scheduler and memory controller enforce CPU, RAM, and I/O limits on container process groups.
- **Overlay2 implements Copy-on-Write using LowerDir and UpperDir:** The Overlay2 storage driver combines read-only image layers (LowerDir) with an ephemeral read-write layer (UpperDir) to form the container root filesystem.
- **Linux Bridge connects container veth pairs:** The host bridge (e.g., docker0) attaches to virtual Ethernet interfaces to provide Layer 2 connectivity between containers.
- **Linux iptables performs NAT for container traffic:** The host kernel uses Destination NAT (DNAT) for inbound port forwarding and Source NAT (SNAT / MASQUERADE) for outbound internet access.
- **Seccomp restricts system calls:** The container runtime applies seccomp profiles to block dangerous Linux syscalls from executing inside container processes.
- **Amazon EKS integrates with containerd:** Managed Amazon EKS worker nodes run containerd as their default container runtime to execute Kubernetes Pods on Amazon EC2 instances.

## 10. Key Takeaways, Entities & Interview Questions

---

### 10.1 Key Takeaways

- Containers are not virtual machines; they are standard Linux processes isolated by Linux Namespaces and resource-constrained by Control Groups (cgroups v2).
- The Open Container Initiative (OCI) standardizes container formats through the OCI Image Specification and OCI Runtime Specification.
- The runtime architecture is layered: Kubernetes Kubelet talks to containerd via CRI, containerd calls runc to create the container, and containerd-shim stays alive to supervise the container process.
- OverlayFS provides storage efficiency using Copy-on-Write (CoW), stacking read-only LowerDir layers beneath an ephemeral read-write UpperDir layer.
- Container security requires a layered defense combining dropped Linux Capabilities, Seccomp system call filtering, and Mandatory Access Control (AppArmor / SELinux).

### 10.2 Important Entities & Classification

- **Core Kernel Primitives:** Linux Namespaces (PID, Mount, Net, IPC, UTS, User, Cgroup), Control Groups (cgroups v1/v2), Completely Fair Scheduler (CFS), OOM Killer.
- **Standards & Protocols:** Open Container Initiative (OCI), Container Runtime Interface (CRI), gRPC, POSIX.
- **Runtime Components:** Docker Engine, containerd, runc, containerd-shim, CRI-O.
- **Storage Components:** OverlayFS, Overlay2, LowerDir, UpperDir, WorkDir, Merged Mount, Copy-on-Write (CoW).

- **Networking Components:** Virtual Ethernet (veth) pair, Linux Bridge (docker0), iptables, nftables, SNAT, DNAT.
- **Security Facilities:** Linux Capabilities, Seccomp, AppArmor, SELinux, Rootless Containers.
- **Cloud & Orchestration Systems:** Kubernetes, Kubelet, Amazon EC2, Amazon EKS, Amazon ECS, AWS Fargate.

### 10.3 Common Technical & Interview Questions

#### 1. What is the fundamental architectural difference between a Container and a Virtual Machine?

*Answer:* A Virtual Machine runs a full guest operating system on top of a hypervisor that virtualizes physical hardware. A container is a standard process that runs directly on the host Linux kernel, using Namespaces for visibility isolation and cgroups for resource limitation without virtualizing hardware.

#### 2. What role does containerd-shim play, and why doesn't runc supervise the container directly?

*Answer:* runc is a short-lived CLI tool that exits immediately after initializing the container to avoid memory overhead. containerd-shim stays alive as the container's parent process to hold exit codes, maintain open I/O pipes, and allow the containerd daemon to restart or upgrade without terminating running containers.

#### 3. How does OverlayFS implement Copy-on-Write when a container modifies an existing image file?

*Answer:* OverlayFS leaves the original file in the read-only LowerDir untouched. It copies the entire file upward into the read-write UpperDir and applies the modification to the copy. Subsequent read operations read the modified version from UpperDir.

#### 4. Why do Kubernetes clusters use containerd directly via CRI instead of running Docker Engine?

*Answer:* Docker Engine introduces unnecessary architectural layers (such as the Docker daemon and REST API) that Kubernetes does not need. Using containerd directly via CRI reduces memory overhead, simplifies the stack, and eliminates the legacy dockershim translation layer.

#### 5. What happens when a container exceeds its memory limit, and which exit code is returned?

*Answer:* When container memory consumption exceeds the cgroup memory.max threshold, the Linux kernel Out-Of-Memory (OOM) Killer terminates the primary process with a SIGKILL signal, resulting in Exit Code 137.